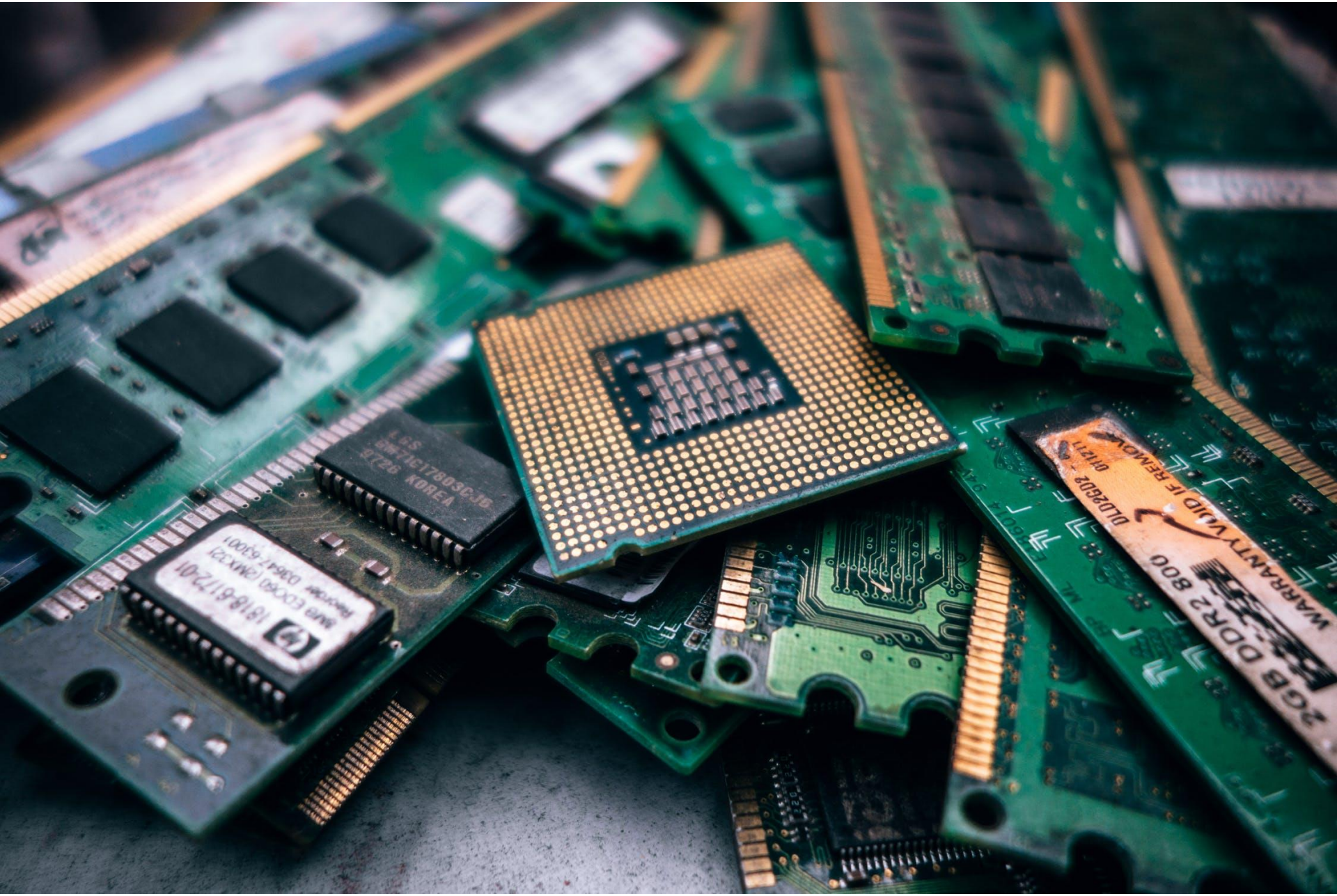


Bild: Fancycrave.com. *Green Ram Card Collection*. 2012-04-15. Pexels.com.

Fysiskt minne

Jimmy Åhlander

2025-11-25

Mittuniversitetet
MID SWEDEN UNIVERSITY

Agenda

- Hårdvara:
  - Separerade minnesrymder.
  - Memory Management Unit (MMU).
- Adressbindning:
  - Att binda minnet i en process.
  - Relocation registers.

- Minnesallokering:
  - Sammanhängande allokering (*contiguous allocation*).
  - Icke-sammanhängande allokering (*non-contiguous allocation*).
    - Paging.

Föreläsningen i ett nötskal

- Den här föreläsningen beskriver det fysiska minnet och därtill relaterade algoritmer.

- Föreläsningen motsvarar kapitel 9 *Main Memory* i Operating System Concepts.

Separerade minnesrymder

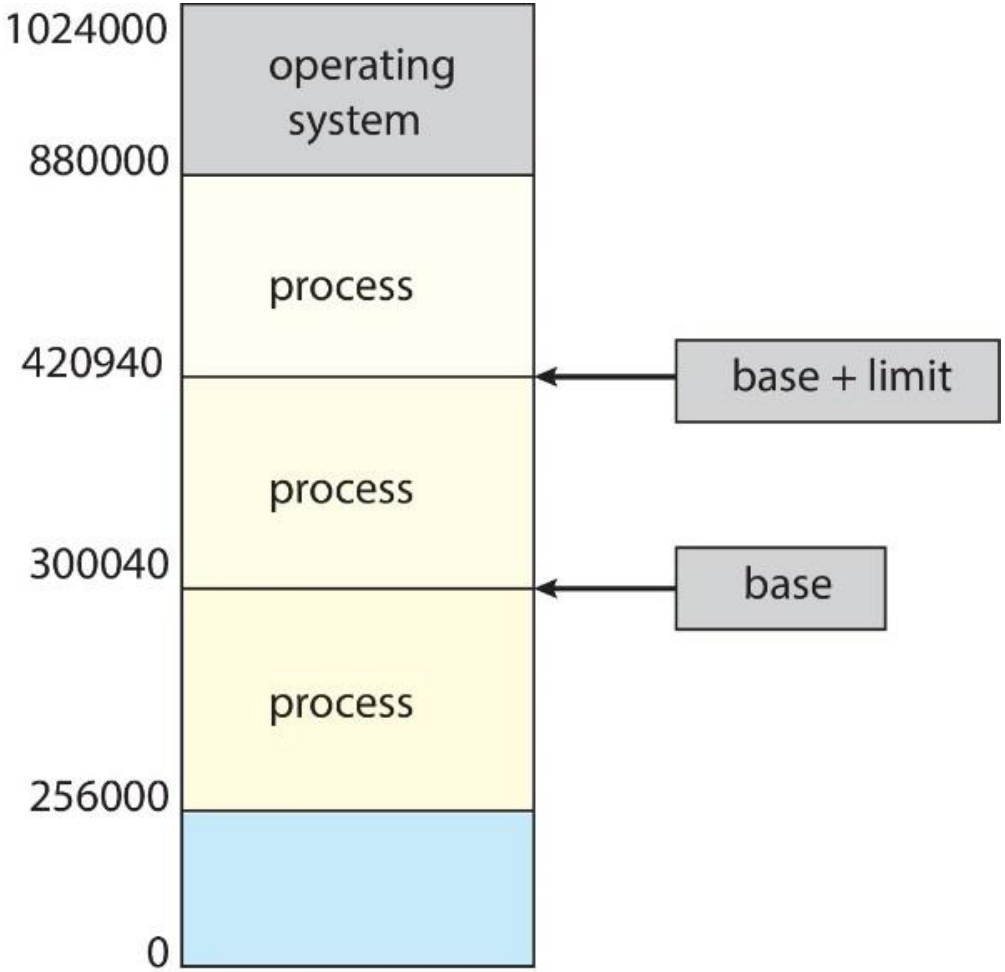


Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 9.01

Grundläggande hårdvara
Memory Management Unit (MMU)

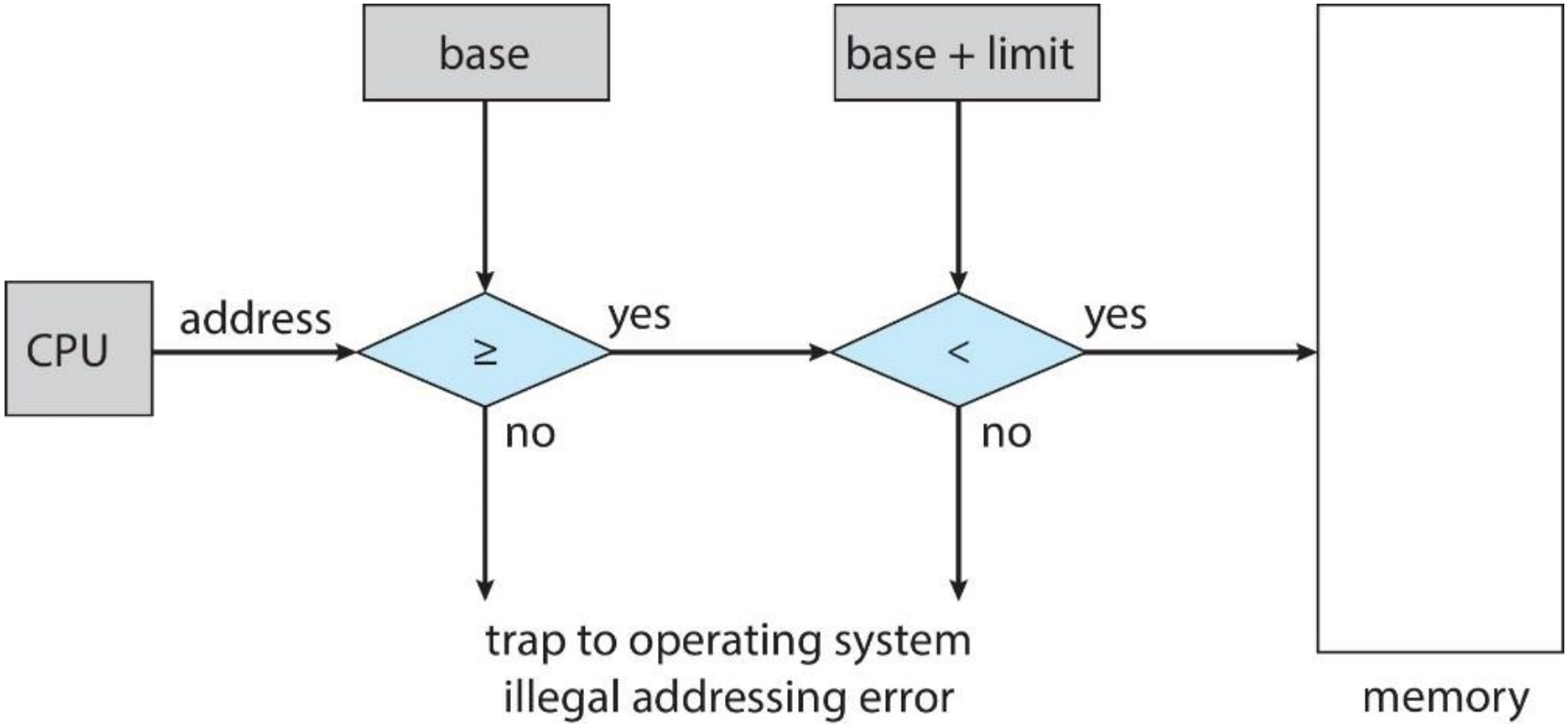


Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 9.02

Att binda minnet i en process

Adressbindning:

- När blir en process bunden till specifika adresser i minnet?

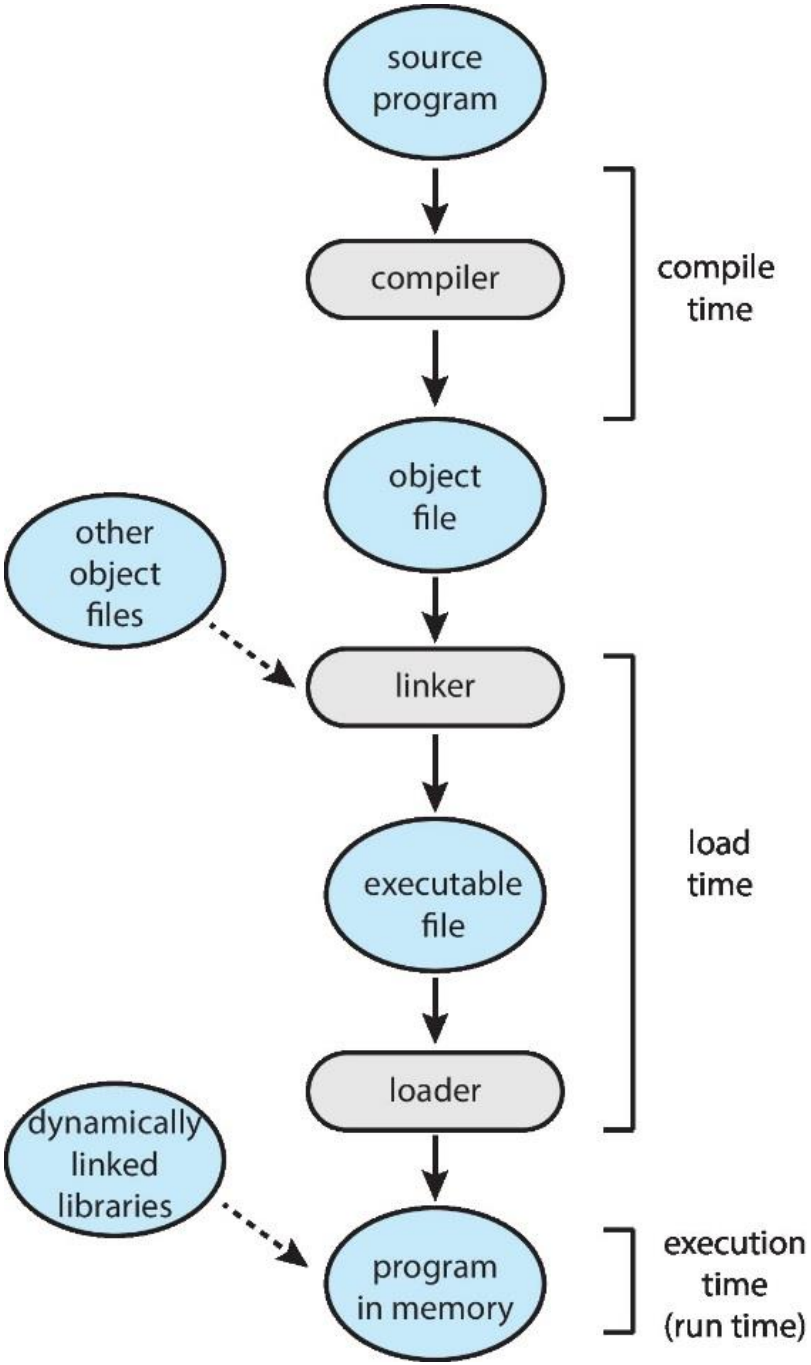


Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 9.03

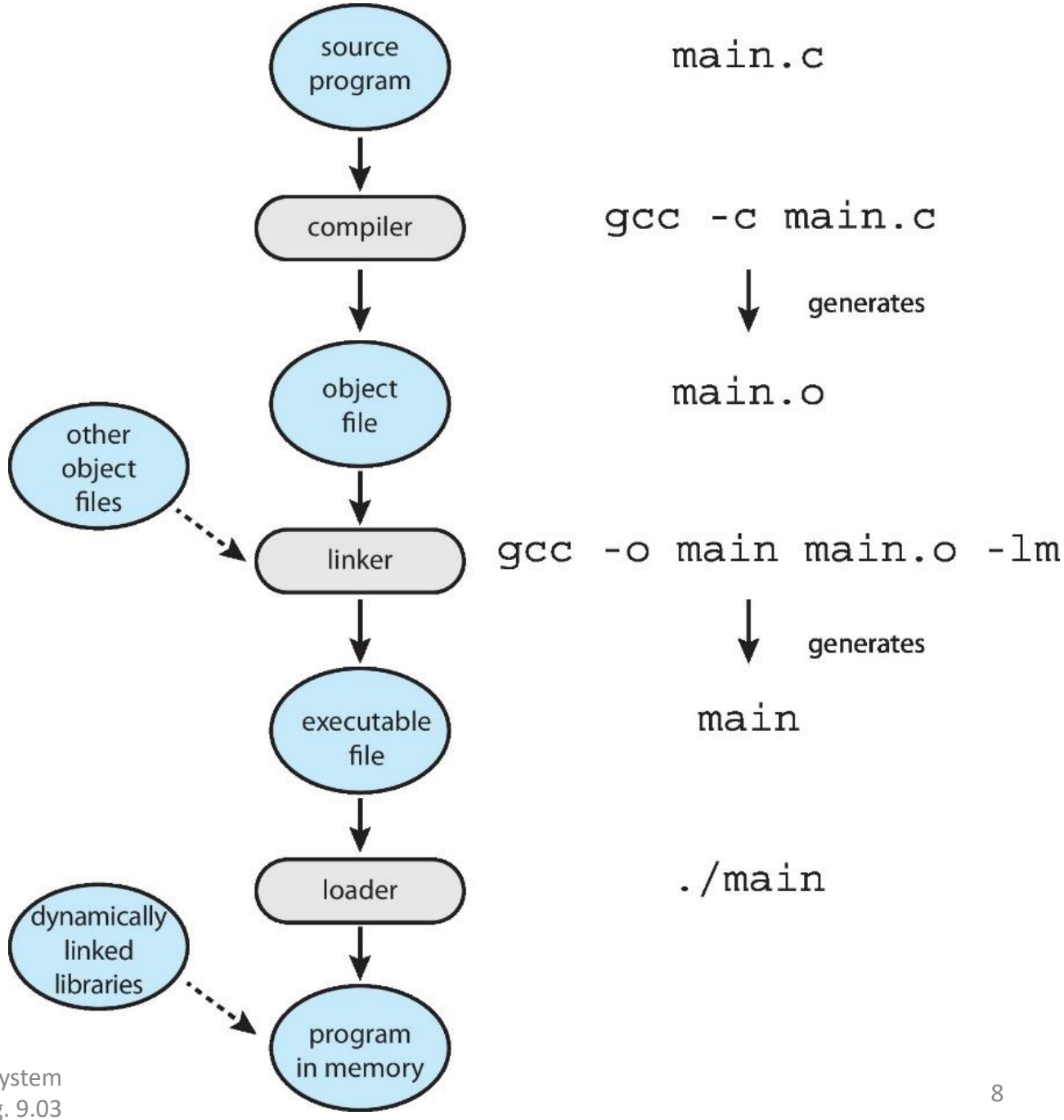


Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 9.03

Logiska och fysiska adresser

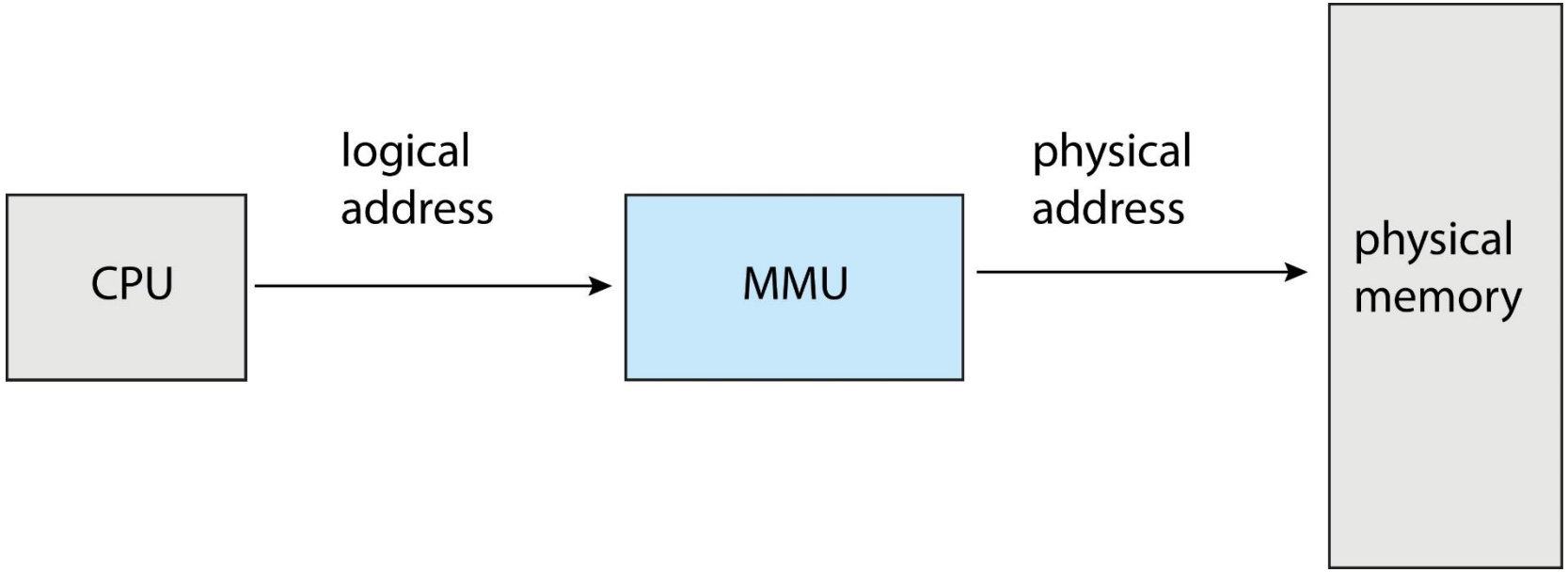


Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 9.04

Relocation registers

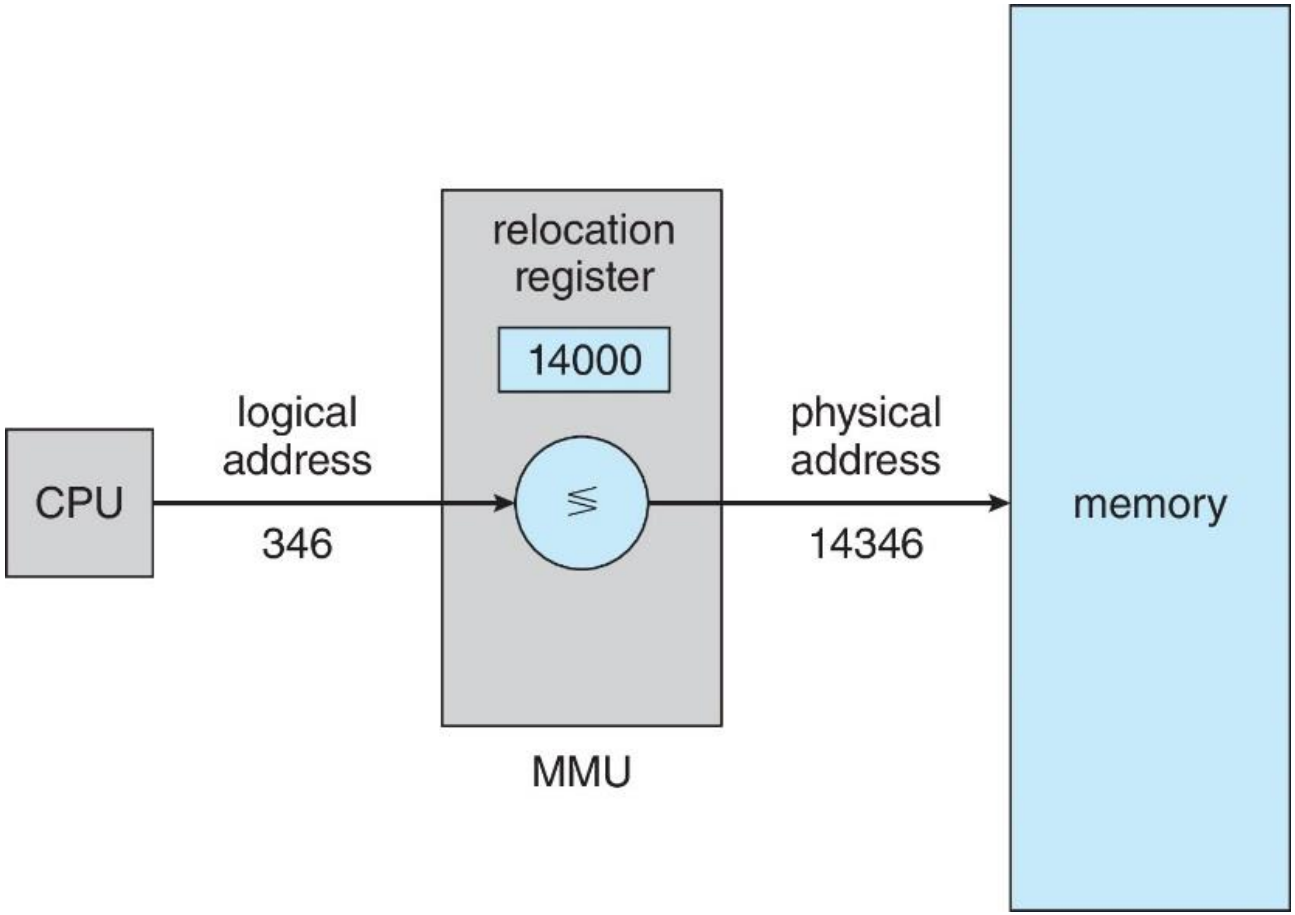


Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 9.05

Relocation registers

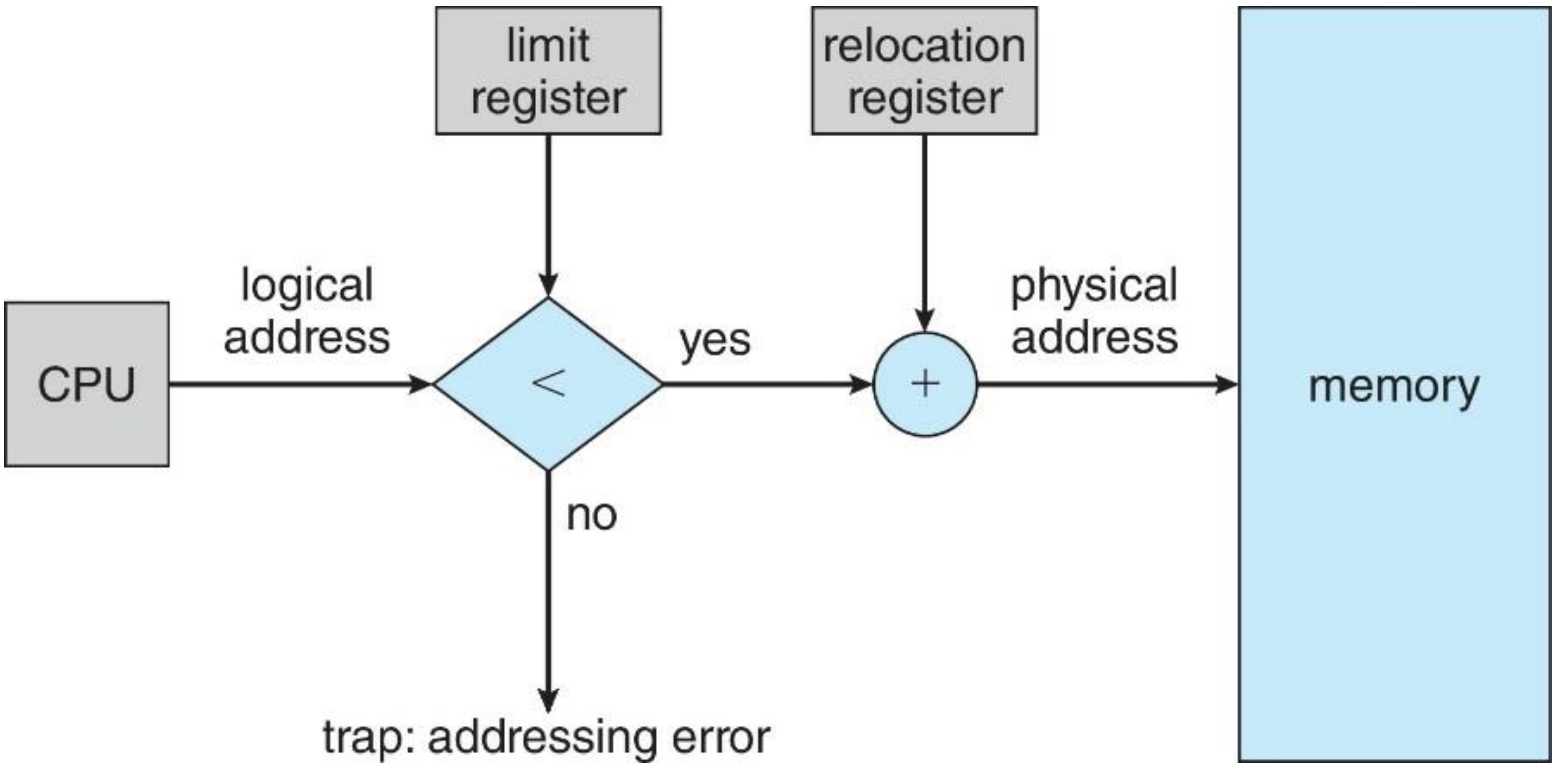


Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 9.06

Minnesallokering

- Sammanhängande allokering (*contiguous allocation*).
- Icke-sammanhängande allokering (*non-contiguous allocation*).
  - Paging.

Sammanhängande allokering (*contiguous allocation*)

- Processer placeras i sammanhängande minnesspann.

- Ger upphov till **extern fragmentering**, och därmed tillhörande
- 50-procentregeln.

Motverka problemet med:

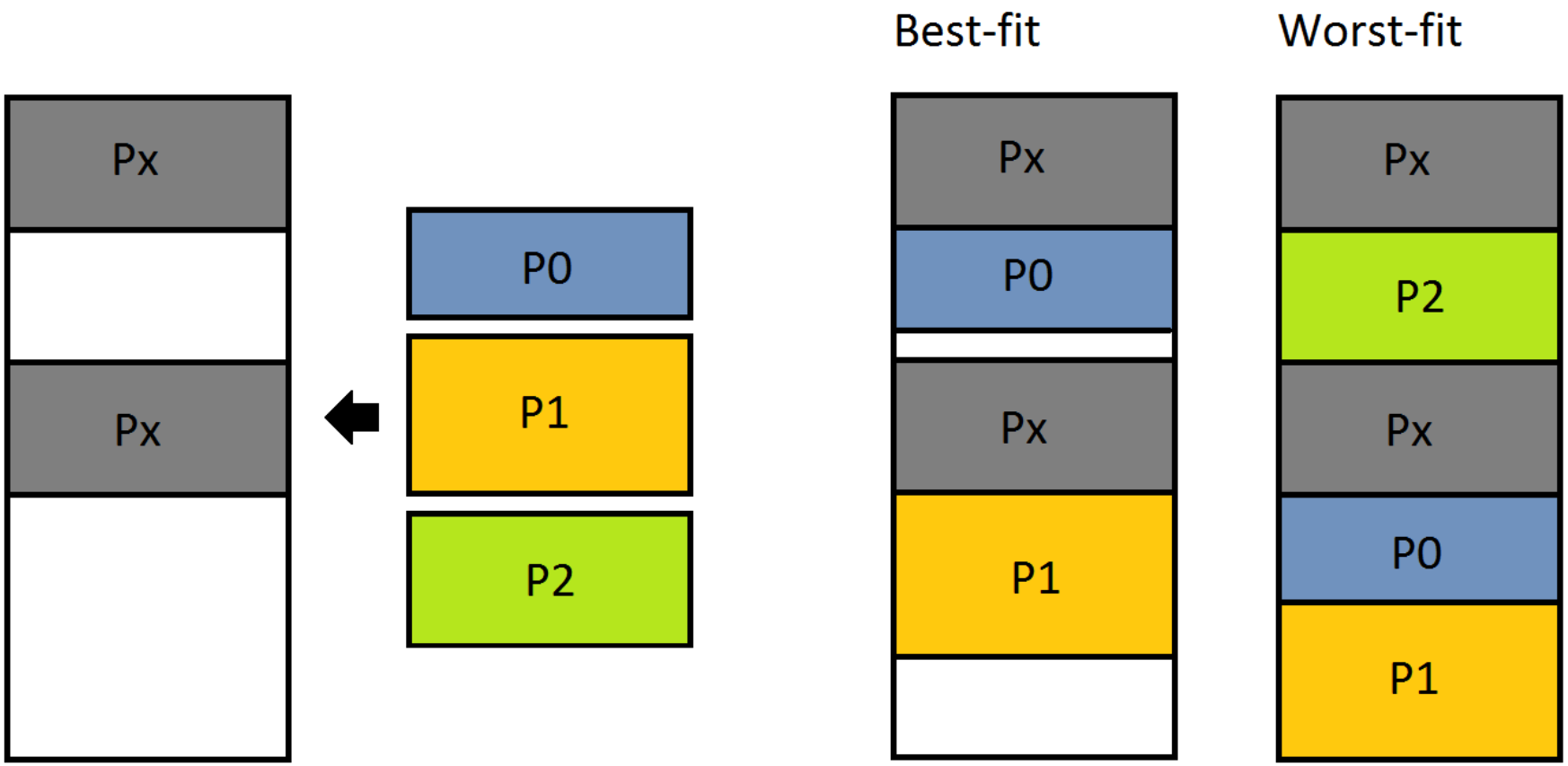
- Kompaktering (*compaction*), och
- Swapping.

Sammanhängande allokering (*contiguous allocation*)

- När det finns flera luckor, hur väljer vi vilken en ny process ska placeras i?

- First-fit.
- Best-fit.
- Worst-fit.

Best-fit vs. Worst-fit



Icke-sammanhängande allokering (*non-contiguous allocation*)

- Paging.

Paging

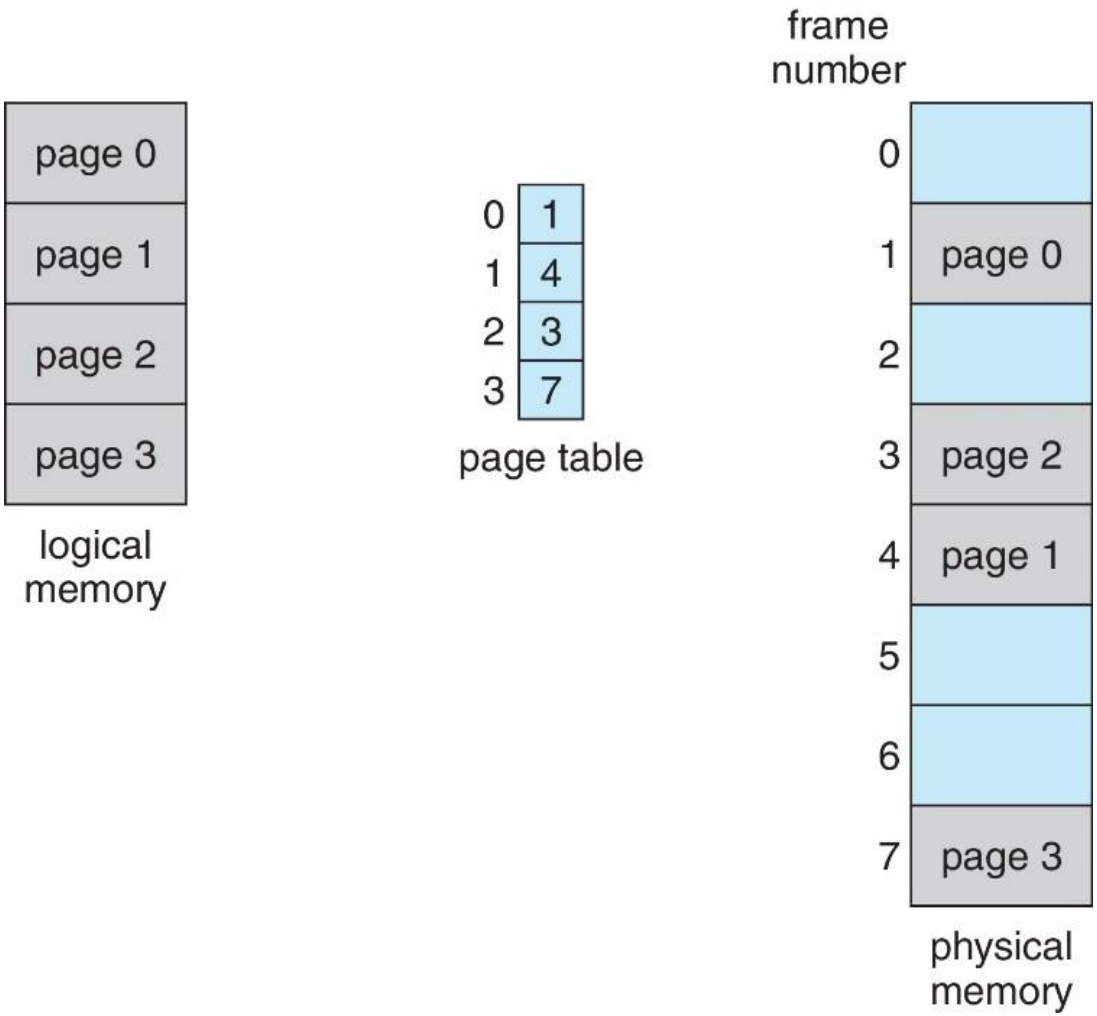


Bild: A. Silberschatz, et al. Operating System Concepts, 9th ed. 2013. Fig. 9.09

Paging

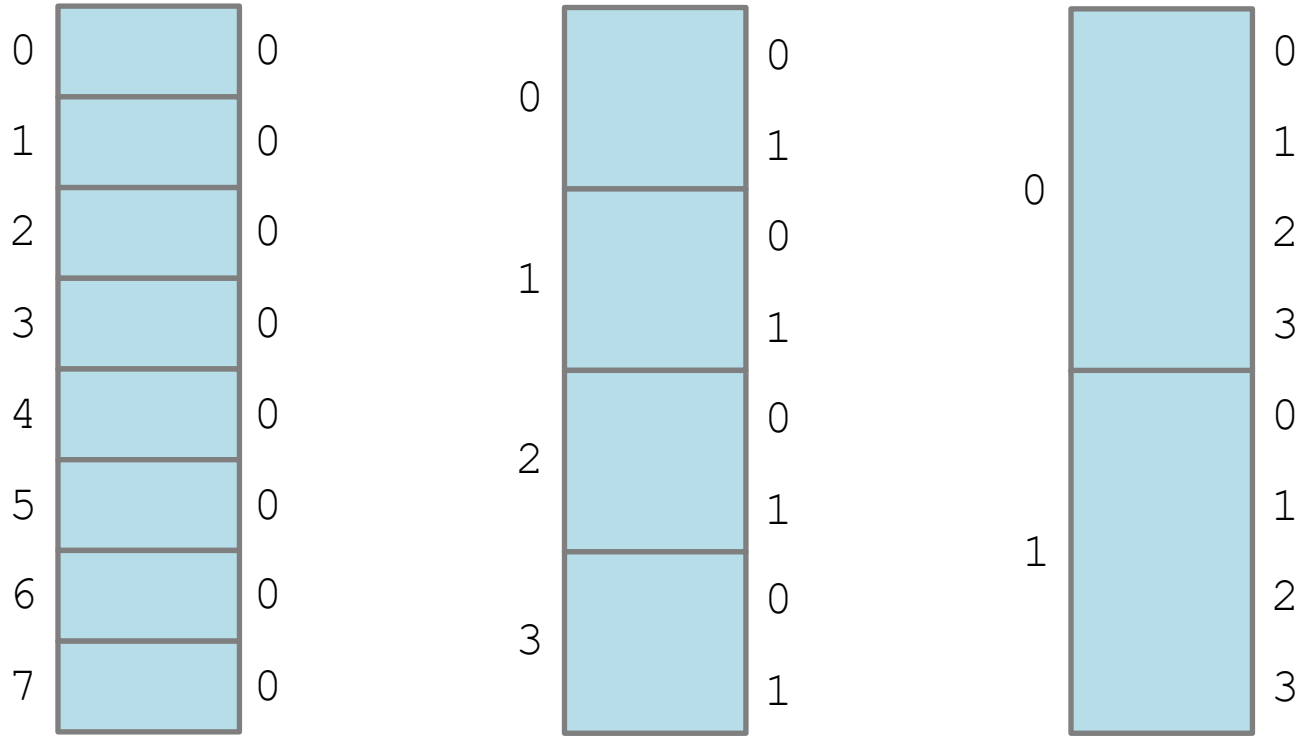
Dela upp den logiska adressen i två delar:

- Sidnummer, och
- Förskjutning (*displacement, offset*).

- Vi har nu sidor med en storlek uttryckt i ett antal bitar av adressen. Med andra ord kommer storleken vara en tvåpotens (2^n).
- T.ex. om sidstorleken är de sista 10 bitarna i en adress blir varje sida
  2^{10} bytes = 1 KiB stor.

Paging

Sidstorlek – sidnummer och förskjutning



pn|d: 000| 00|0 0|00

Paging

- Storleken på en sida måste vara exakt lika stor som storleken på den fysiska motparten: ramen (*frame*). Ramstorleken och därmed effektivt även sidstorleken avgörs av hårdvaran.

- Varje sida kopplas till en ram.
- Kopplingarna sparas i en sidtabell (*page table*).

Paging
Hur en sida kopplas till en ram

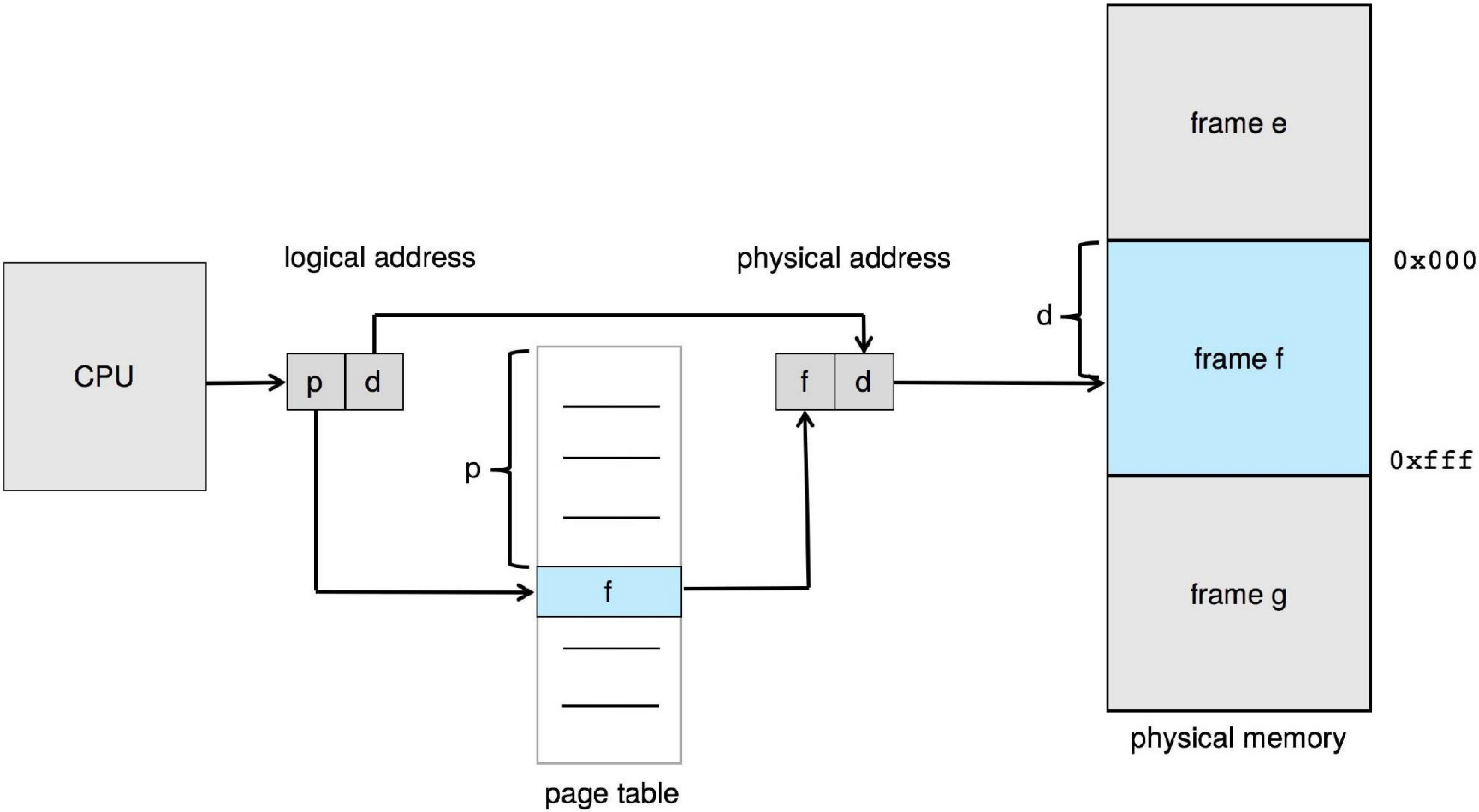


Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 9.07

Paging
Sidtabeller

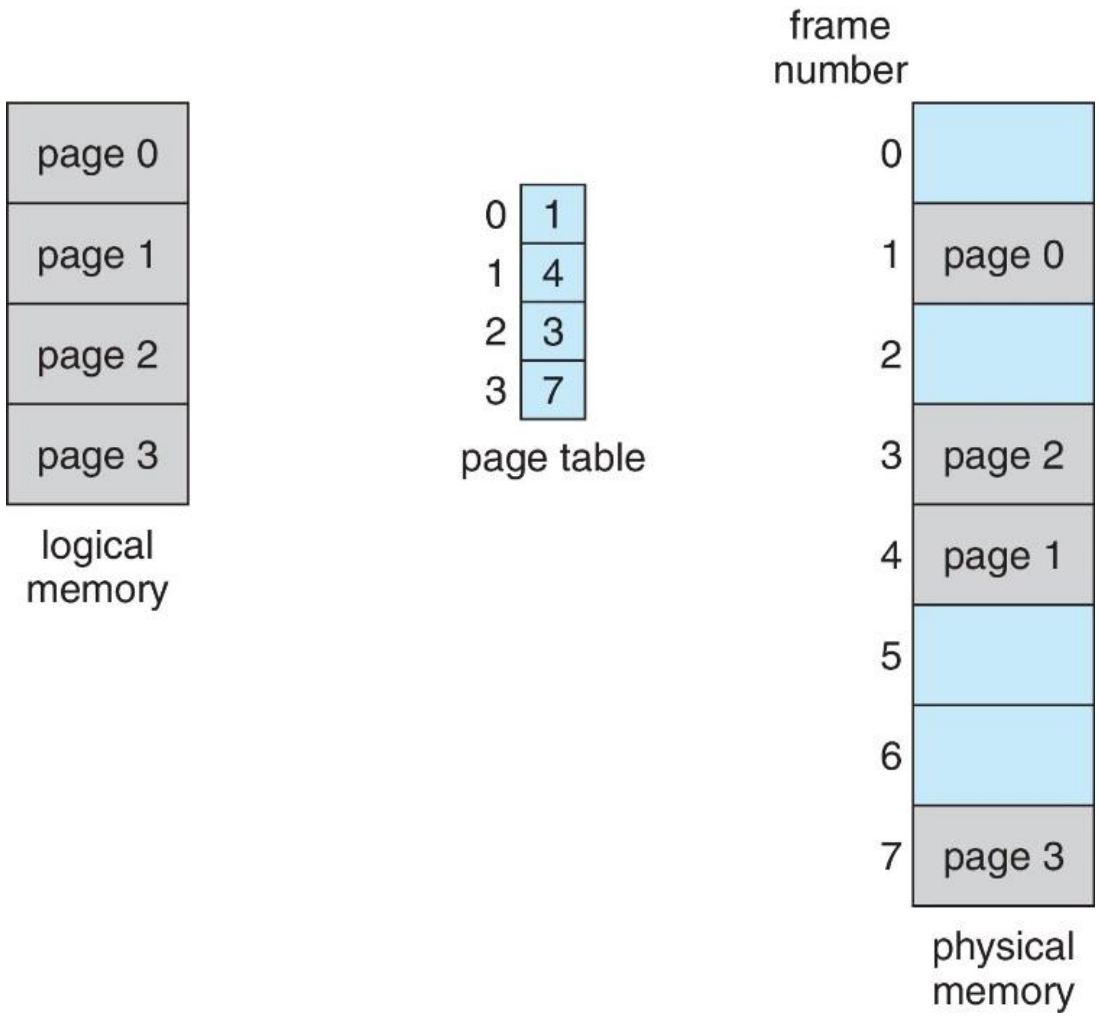


Bild: A. Silberschatz, et al. Operating System Concepts, 9th ed. 2013. Fig. 9.09

Paging Exempel

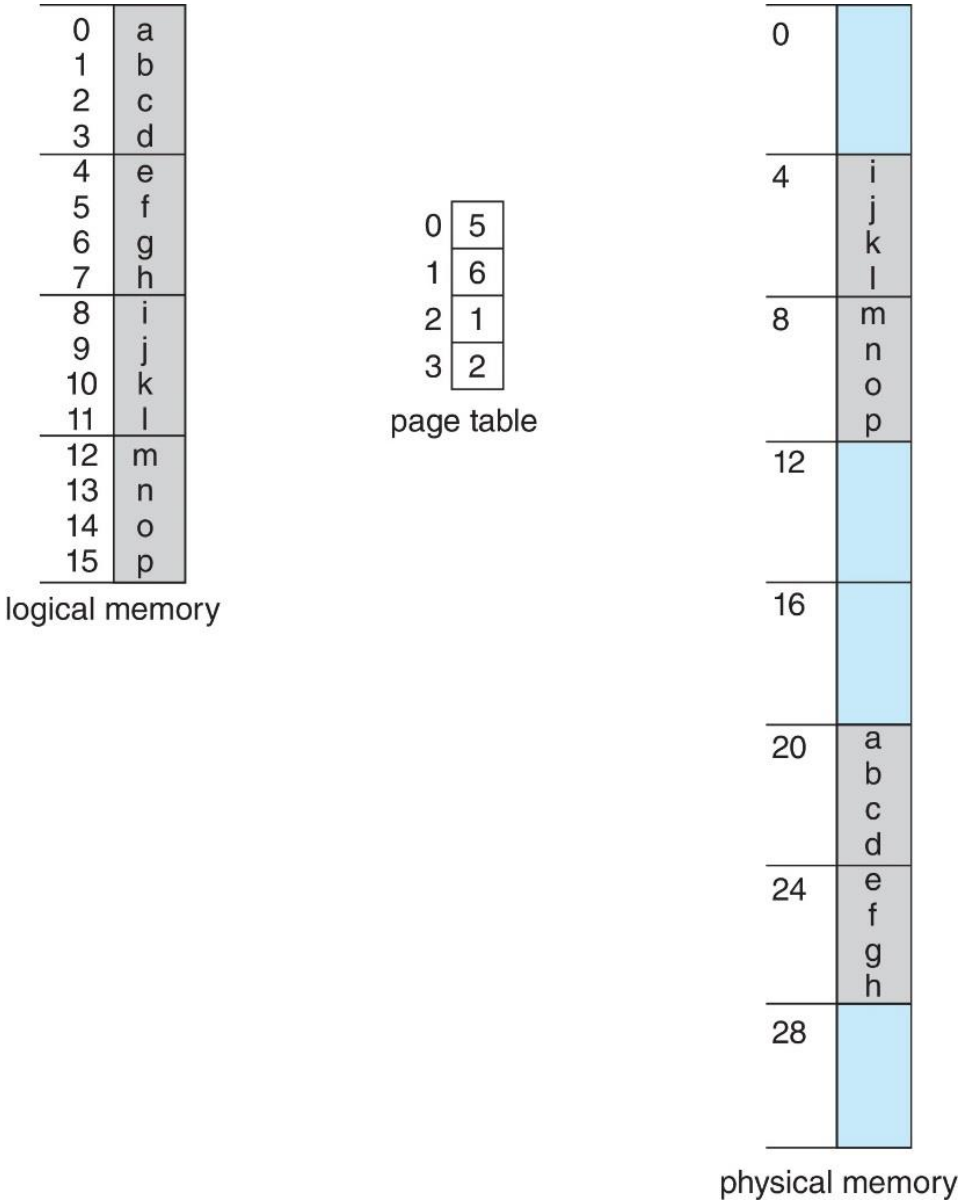


Bild: A. Silberschatz, et al. Operating System Concepts, 9th ed. 2013. Fig. 9.10

Att hålla reda på lediga ramar – ett exempel

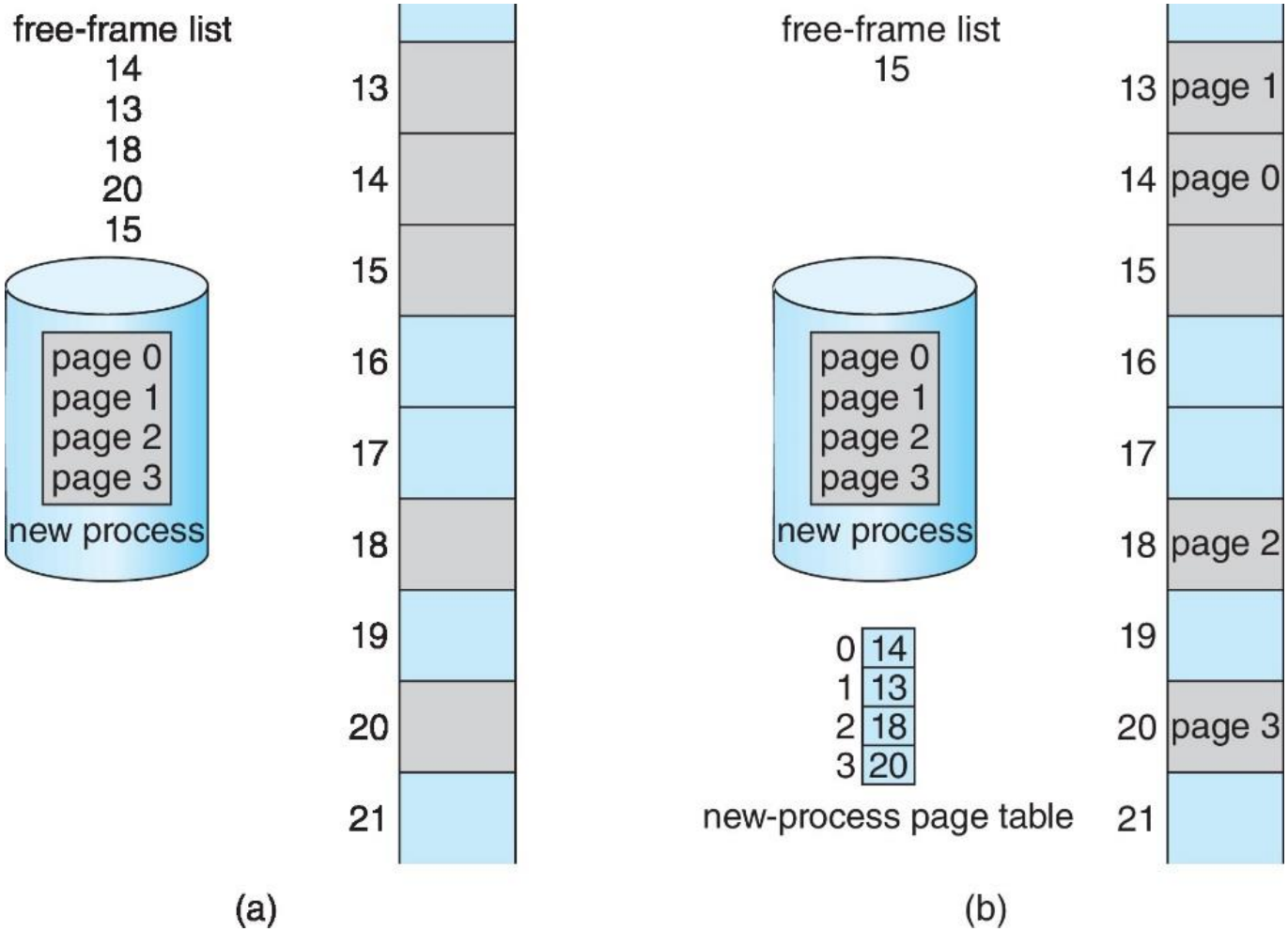
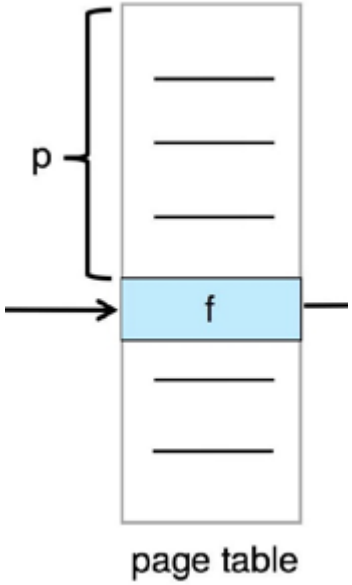


Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 9.11

Paging
Sidtabellerna

- Sidtabellen sparas generellt av OS i processens PCB.

- Dispatchern kan ladda processens sidtabell till hårdvarucaches om sidtabellen är tillräckligt liten.
- Större sidtabeller måste lagras i primärminnet. Vi lagrar då i PCB endast en pekare som anger var i primärminnet tabellen är lagrad.
- Detta kräver dock en extra minnesåtkomst. *Med andra ord kommer varje minnesreferens kräva två åtkomster*!
- Lösningen är att använda en *translation look-aside buffer* (TLB) – en mycket snabb hårdvarucache.



Paging med en TLB

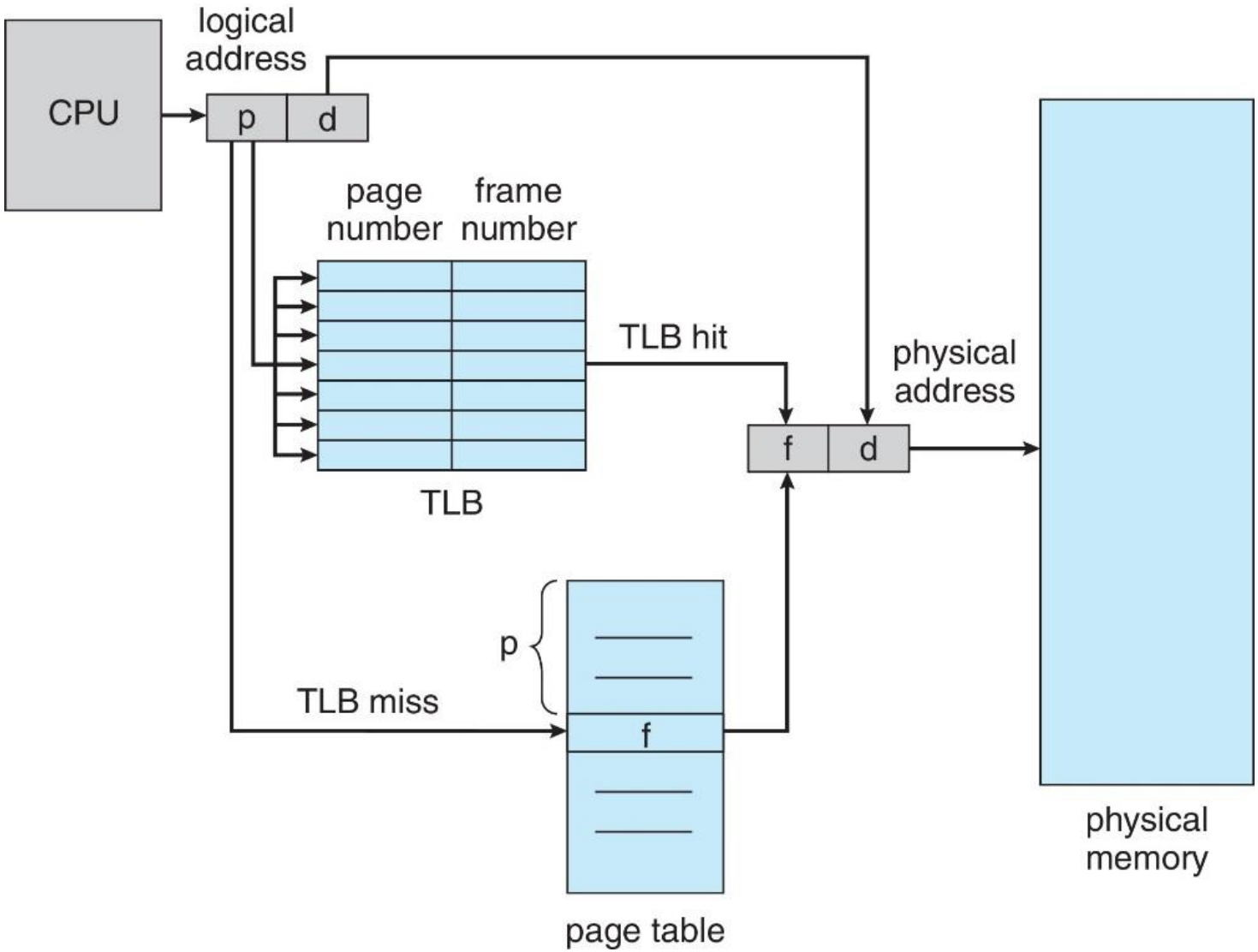


Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 9.12

Translation look-aside buffer (TLB)

- Vissa TLBs har *address-space identifiers* (ASIDs).
- ASIDs gör att en TLB kan hantera sidor från flera olika processer samtidigt.
- På så sätt behöver inte TLBn rensas vid varje context switch.

Effektiv åtkomsttid med paging och en TLB

- Paging medför en tidskostnad.
- Det går att beräkna den effektiva åtkomsttiden till minnet.
- Effektiv åtkomsttid till minnet (*effective memory-access time*) är medeltiden varje minnesåtkomst tar, med hänsyn till hur sannolikt det är att vi får en träff eller miss i TLBn.

Effektiv åtkomsttid med paging och en TLB

Effective memory-access time

We can compute the effective access time t by

$$t = (1 - p) \times (t_t + t_m) + p \times (t_t + t_m + t_m),$$

where p is the probability of a TLB miss, t_t is the time for a TLB lookup, t_m is the time for a memory access.

Paging
Skydd

- Vad händer när en sidtabell är större än nödvändigt för en process?
  - Vi måste skydda sidorna i en process.
- Vad händer när en process behöver minne som inte är en exakt multipel av sidstorleken?
  - Vi riskerar **intern fragmentering**.

- Ponera ett system med en 14-bitars adressrymd och en process med de logiska adresserna 0 till och med 10 468. Sidstorleken är 2 KiB.

  ...

Paging
valid och invalid bits

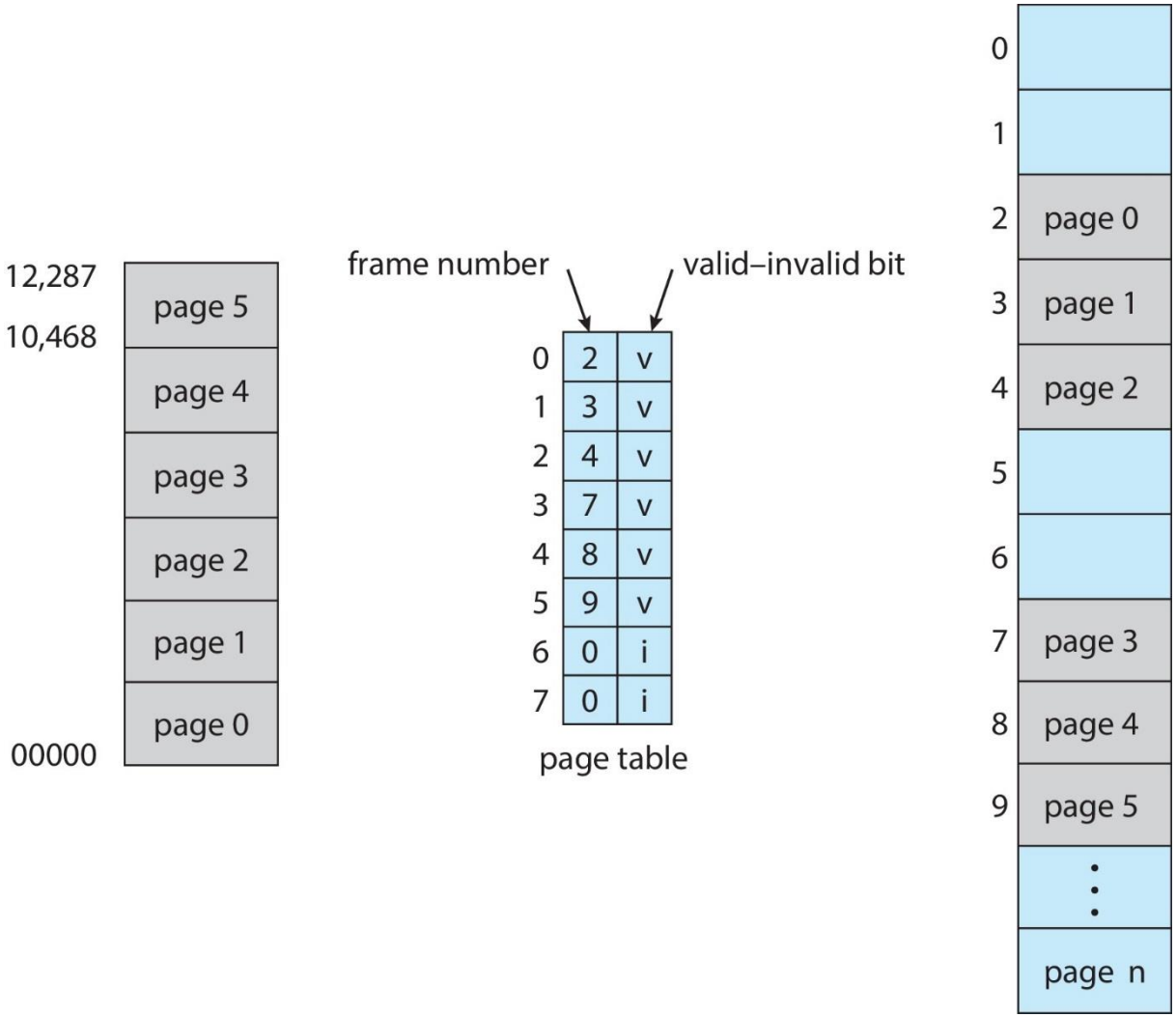


Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 9.13

Paging
Fler bitar

- Vi kan också lägga till fler bitar i sidtabellen för andra ändamål.
  - T.ex. för att specificera om läs- eller skrivoperationer är tillåtet, eller om exekveringen är tillåtet på olika sidor.

- På så sätt kan exekveringsskydd implementeras för vissa minnesregioner, t.ex. där webbläsare lagrar data som har laddats ned från webben.

- Vidare skulle processer kunna dela sidor med varandra som endast får läsas (*read-only*) och på så sätt spara primärminne.

Paging - Delade sidor

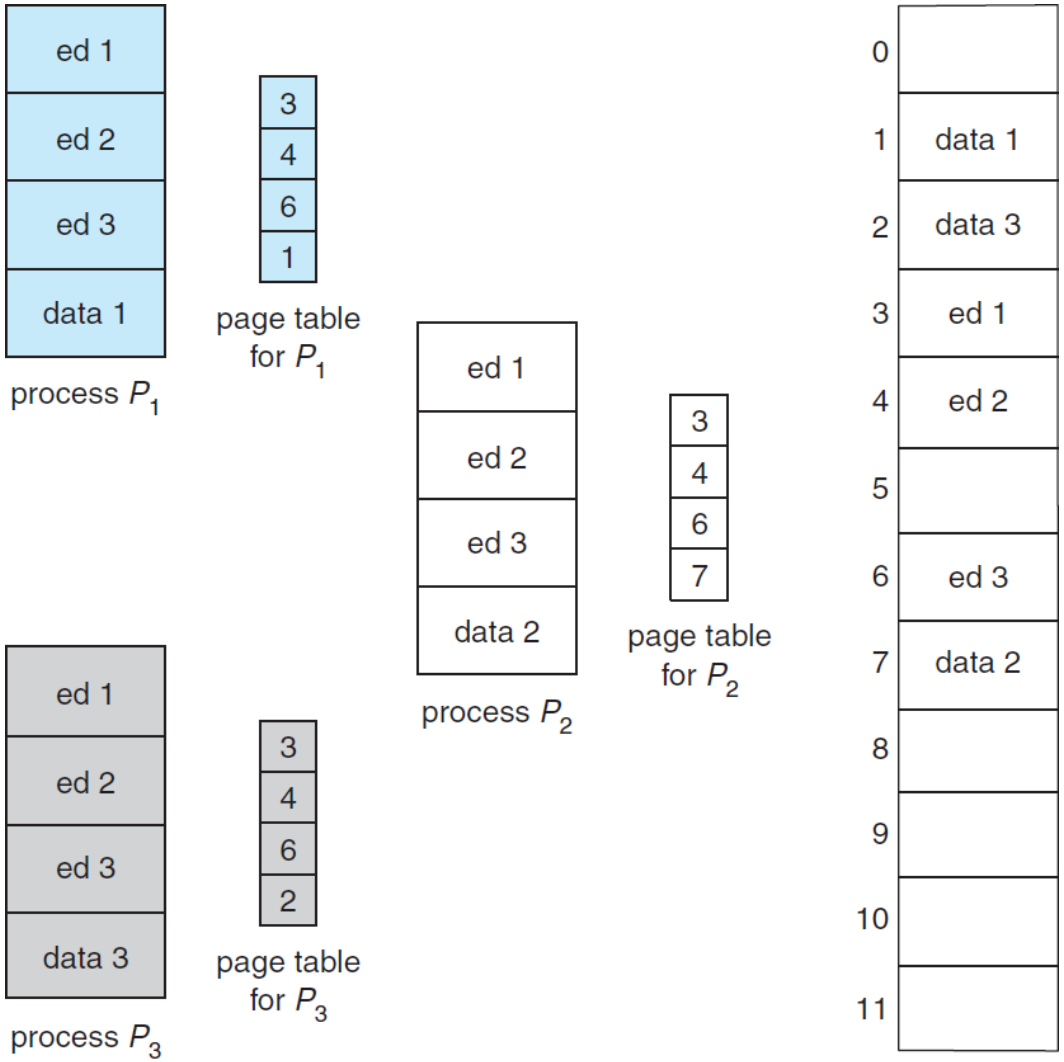


Bild: A. Silberschatz, et al. Operating System Concepts, 9th ed. 2013. Fig. 8.16

Paging
Enorma adressrymder

- Vad händer om vi har enorma logiska adressrymder? T.ex. 32-bitars eller 64-bitars adressrymder.

- Med en sidstorlek på 4 KiB (d.v.s. 12 bitar av adressen) är worst-case scenario i ett 32-bitars system att sidtabeller blir 4 MiB stora – för varje process!

- Om vi måste allokera allt utrymme för dessa sidtabeller sammanhängande är vi tillbaks på ruta 1.
  - En lösning kan vara att använda paging på själva sidtabellen. Paging på paging. Kallas för *hierarkisk paging*.

Paging
Enorma adressrymder

- Hierarkisk paging kanske funkar för 32-bitars system men inte för 64-bitars system.

Alternativ inkluderar:

- Hashfunktioner.
- Inverterade sidtabeller.
  - Ett inlägg per ram istället för ett inlägg per sida.
  - Vi måste då också lagra vilken process som äger vilken ram.
  - Vi måste söka igenom sidtabellen för att hitta korrekt inlägg.

Paging
Inverterad sidtabell

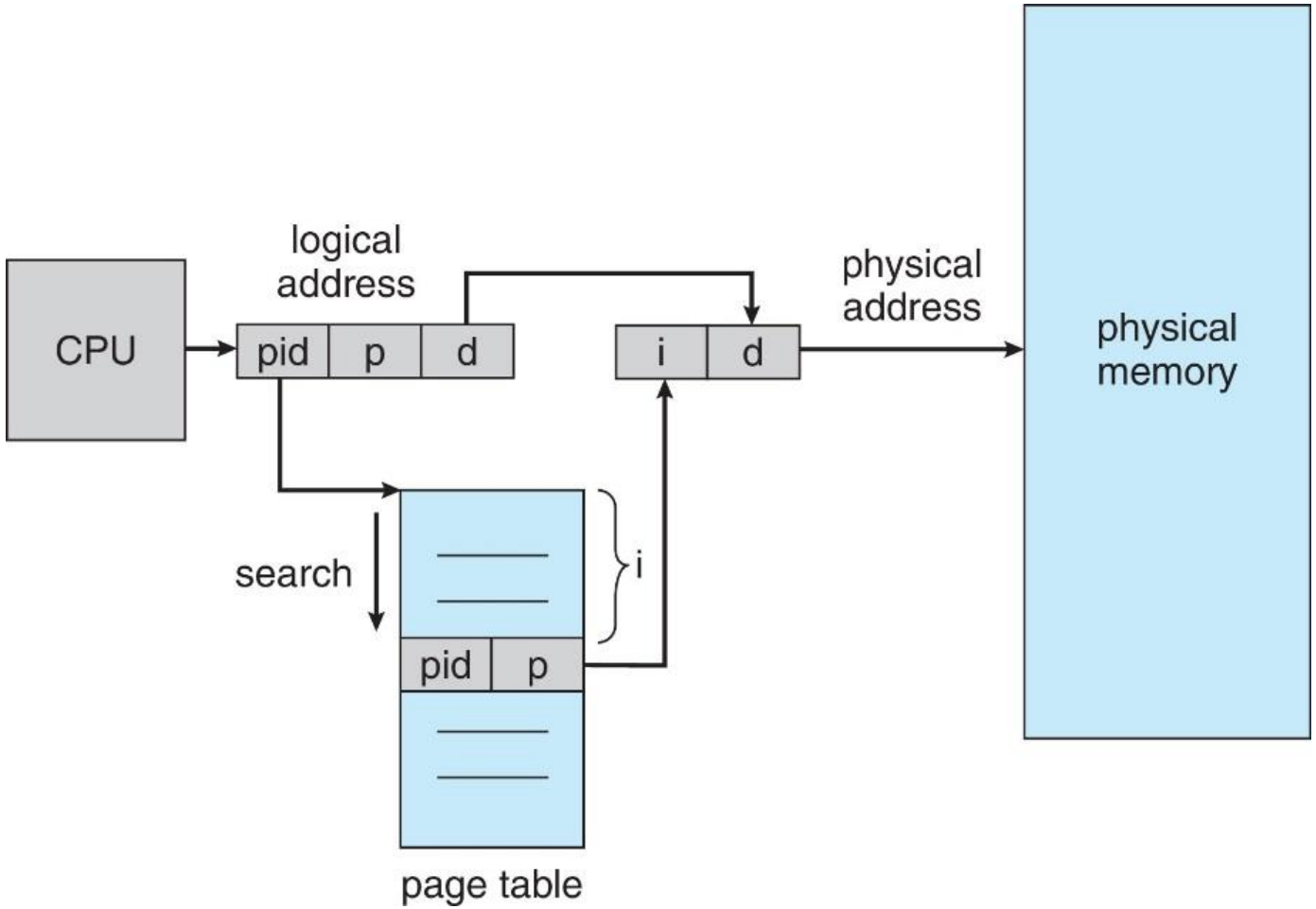


Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 9.18

Paging i ett nötskal

- Vi vill undvika extern fragmentering. Intern fragmentering är inte lika kostsam.
- Sidtabeller blir ofta stora och måste lagras i primärminnet. Den effektiva minnesåtkomsttiden ökar vilket till viss del kompenseras av en hårdvarucache: en TLB.
- Stora sidor ger mer intern fragmentering. Små sidor ger större sidtabeller.

Fysiskt minne
Sammanfattning

- Processer måste placeras någonstans i det fysiska minnet. När adressbindningen sker varierar.
- MMU:n ansvarar för att processer håller sig inom sina adresspann och översätter logiska adresser till fysiska.
- Sammanhängande allokering är enkelt men lider av extern fragmentering.
- Icke-sammanhängande allokering som paging löser extern fragmentering men lider av sämre effektiv åtkomsttid till minnet och intern fragmentering.